Learning C#/WPF From The Polynomial Trajectory Planner

Purpose

This document defines the starting learning path for the `Learning C#/WPF` activity.

The path is based on the existing application in `project/src/`, but it does **not** treat the whole project as a single block to absorb all at once. The project is used as a real reference system, while the lessons isolate one idea at a time and then reconnect it to the full application.

Why This App Is A Good Teacher

The existing solution already has a strong educational shape:

- `TrajectoryPlanner.App`: a WPF desktop application.
- `TrajectoryPlanner.Core`: a calculation library with no UI dependency.
- `TrajectoryPlanner.Core.Tests`: tests focused on the mathematical layer.

This separation is excellent for learning because it exposes three distinct questions:

1. What is C# as a language?
2. What is WPF as a UI framework?
3. How should a desktop app be divided into responsibilities?

The current app is also useful because it is **formal enough to be real**, but still **simple enough to inspect**. It is not yet the final architecture of the project, and that is a strength for learning: we can study both solid decisions and provisional ones.

What I Observed In The Existing App

The following elements are particularly important for teaching:

- The solution file is small and clear: one WPF project, one core library, one test project.
- The WPF app currently uses a **prototype-style structure** centered on `MainWindow.xaml`, `MainWindow.xaml.cs`, `PrototypeModels.cs`, and `PrototypeTrajectoryService.cs`.
- The app uses `DataContext = this`, `INotifyPropertyChanged`, `ObservableCollection<T>`, event handlers, and XAML bindings. This makes it very suitable for introducing practical WPF fundamentals.
- The core library already shows good conceptual separation: segments, trajectories, states, evaluators, builders, constraints, blending, and tests.
- The planning documents confirm that the prototype is intentionally a **validation testbed**, not the final refined application architecture.

What We Should Not Do First

We should **not** start by reading the whole project from top to bottom.

That would expose too many ideas at once:

- C# syntax
- records and classes
- property change notification
- XAML layout
- binding
- event-driven UI
- chart integration
- mathematical domain logic
- solution architecture

All of those are legitimate, but taken together they create confusion instead of understanding.

How We Should Begin

We should begin from a **single vertical slice** of the existing app:

1. `App.xaml`
2. `MainWindow.xaml`
3. `MainWindow.xaml.cs`
4. `PrototypeModels.cs`

This slice lets us answer the first truly important question:

> How does a C#/WPF desktop window become a living interactive object?

That question is better than starting from syntax alone, because it immediately gives purpose to the language constructs:

- a class exists to model behavior;
- a property exists to expose state;
- `INotifyPropertyChanged` exists so the UI can react;
- XAML exists to declare structure;
- event handlers exist to connect user actions to code.

First Teaching Decision

The first phase of the learning path will **not** be "advanced trajectory logic".

It will be:

Phase 1 — Read The WPF Prototype As A Living System

Goal

Understand how the current application window is assembled and why it reacts to user input.

Focus

- what `App.xaml` does;
- what a `Window` is;
- what XAML declares and what code-behind performs;
- what `DataContext` means;
- what binding means in practice;
- why `ObservableCollection<T>` is used;
- why `INotifyPropertyChanged` is required;
- how button clicks trigger computation;
- how computed results are pushed back into the UI.

Why This Is The Right Start

This phase gives immediate contact with a real application while staying inside a manageable boundary. It also creates the vocabulary needed for every later step.

Without this phase, later topics such as MVVM, services, or refactoring would be only words.

Learning Sequence

Stage A — C# For This App

We will study only the C# features that are immediately active in the app:

- classes
- records
- properties
- fields

- constructors
- methods
- collections
- events
- exceptions
- namespaces
- access modifiers

These will be taught from the project itself, not from unrelated examples unless simplification becomes necessary.

Stage B — WPF Fundamentals

We will study:

- XAML structure
- controls
- layout containers
- bindings
- converters
- `DataContext`
- code-behind
- event-driven interaction

At this stage, the goal is not "purity". The goal is to understand what the current application is doing.

Stage C — UI And Domain Separation

After the basics are clear, we will examine why:

- `MainWindow` should not contain all logic forever;
- `PrototypeTrajectoryService` is a useful intermediate step;
- `TrajectoryPlanner.Core` is the real computational heart;
- tests exist in a separate project.

This is where we will start distinguishing:

- what is acceptable in a prototype;
- what is better in a long-term application;
- what should eventually move toward a stricter MVVM structure.

Stage D — Controlled Reconstruction

Once the existing app is intelligible, we will build small exercises and mini-apps inside `LearningCsWPF/apps/`.

Those exercises should progressively reconstruct the same ideas in simpler contexts:

1. a minimal window with one bound property;
2. a window with one input, one button, one output;
3. a window with a small model object and property change notification;
4. a window that calls a service;
5. a simplified trajectory viewer.

Only after that will the full project feel readable instead of opaque.

What Is Solid Vs Provisional In The Existing App

Solid enough to learn from immediately

- solution/project separation;
- C# project references;
- core library independent from WPF;
- use of tests for the core;

- WPF bindings and property change notification;
- event-to-service-to-result flow;
- domain naming tied to the theory documents.

Provisional or intentionally not final

- the prototype-style concentration of many responsibilities in `MainWindow`;
- the use of `DataContext = this` in the main window;
- the absence of a stricter MVVM layer with dedicated view-model classes and commands;
- the current exact pass-through implementation, which project notes already mark as provisional.

This distinction matters pedagogically: we must learn from the app without silently treating every current choice as the final ideal.

First Lesson

The first lesson should be:

Lesson 1 — How The Main Window Lives

Material

- `project/src/TrajectoryPlanner.App/App.xaml`
- `project/src/TrajectoryPlanner.App/MainWindow.xaml`
- `project/src/TrajectoryPlanner.App/MainWindow.xaml.cs`
- `project/src/TrajectoryPlanner.App/PrototypeModels.cs`

Questions We Will Answer

1. What starts the application?
2. How does WPF know which window to open?
3. What is declared in XAML and what is executed in C#?
4. Why are some UI values written as `{Binding ...}`?
5. How does changing a property update the screen?
6. Why does the segment list react when elements are added or removed?
7. What exactly happens when the user clicks `Compute`?

Expected Result

At the end of Lesson 1, you should be able to describe the current app's interaction loop in plain words, without yet needing to understand the full mathematical core.

Teacher's Starting Position

I will teach from the existing app, but I will not force you to admire the project as a finished monument.

We will treat it as:

- a real machine,
- a partially provisional design,
- a laboratory for learning judgment.

That is the right attitude for this material.

Next Step

Begin with **Lesson 1 — How The Main Window Lives**.

If desired, the next artifact to create in this learning workspace should be a short guided lesson note dedicated only to that first vertical slice, followed by a tiny WPF exercise app that reproduces the same interaction pattern in a much smaller form.